

Week 1 — Practice Problems — Solutions

Day 1 Live-Coding Session — 5 Problems

Name: Gursimran Bajwa | **Registration No:** RA2511003011289**Language:** Java | **Problems solved:** 5 / 5

Problem 1 — Rock-Paper-Scissors Game

Problem Statement

The College Coding Arcade. The coding club needs a simulator that plays several Rock-Paper-Scissors rounds between a player and the computer, records the outcome of every round, and prints a final scoreboard with wins, losses, draws and the player's win percentage.

Approach

- The computer's move is picked at random from a MOVES array using `Random.nextInt(3)`. A fixed seed is used so the demo run is reproducible; removing the seed makes it fully random.
- `playRound()` applies the standard rules: equal moves are a draw, otherwise the three winning combinations for the player are checked with a single boolean expression.
- Player moves, computer moves and results are stored in three parallel arrays so the round table can be reprinted as a summary after all rounds finish.
- Wins, losses and draws are counted inside the loop, and the win percentage is computed as $(\text{wins} * 100.0) / \text{rounds}$, printed to one decimal place.

Java Program — RockPaperScissorsGame.java

```
import java.util.Random;

/**
 * Problem 1 - Rock-Paper-Scissors Game (The College Coding Arcade)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class RockPaperScissorsGame {

    static final String[] MOVES = { "Rock", "Paper", "Scissors" };

    /* Returns the outcome of one round from the player's point of view. */
    public static String playRound(String playerMove, String computerMove) {
        if (playerMove.equalsIgnoreCase(computerMove)) {
            return "Draw";
        }
        boolean playerWins =
            (playerMove.equalsIgnoreCase("Rock")    && computerMove.equalsIgnoreCase("Scissors")) ||
            (playerMove.equalsIgnoreCase("Paper")   && computerMove.equalsIgnoreCase("Rock"))    ||
            (playerMove.equalsIgnoreCase("Scissors") && computerMove.equalsIgnoreCase("Paper"));
        return playerWins ? "Player Wins" : "Computer Wins";
    }

    public static void main(String[] args) {
        int rounds = 5;
        // Predefined player moves for the live demo (replace with Scanner input if needed).
        String[] playerMoves = { "Rock", "Paper", "Scissors", "Rock", "Paper" };
        String[] computerMoves = new String[rounds];
        String[] results = new String[rounds];

        Random random = new Random(2); // fixed seed -> reproducible demo run
        int wins = 0, losses = 0, draws = 0;

        for (int i = 0; i < rounds; i++) {
            computerMoves[i] = MOVES[random.nextInt(3)];
            results[i] = playRound(playerMoves[i], computerMoves[i]);
        }
    }
}
```

```

        if (results[i].equals("Player Wins")) wins++;
        else if (results[i].equals("Computer Wins")) losses++;
        else draws++;

        System.out.println("Round " + (i + 1) + " - Player: " + playerMoves[i]
            + ", Computer: " + computerMoves[i] + " -> " + results[i]);
    }

    System.out.println();
    System.out.println("----- FINAL SCOREBOARD -----");
    System.out.printf("%-8s %-14s %-16s %-14s%n", "Round", "Player Move", "Computer Move", "Result");
    for (int i = 0; i < rounds; i++) {
        System.out.printf("%-8d %-14s %-16s %-14s%n",
            (i + 1), playerMoves[i], computerMoves[i], results[i]);
    }
    System.out.println("-----");

    double winPercentage = (wins * 100.0) / rounds;
    System.out.println("Wins: " + wins + " | Losses: " + losses + " | Draws: " + draws
        + " | Win % = " + String.format("%.1f", winPercentage) + "%");
    }
}

```

Output

```

Round 1 - Player: Rock, Computer: Paper -> Computer Wins
Round 2 - Player: Paper, Computer: Rock -> Player Wins
Round 3 - Player: Scissors, Computer: Scissors -> Draw
Round 4 - Player: Rock, Computer: Paper -> Computer Wins
Round 5 - Player: Paper, Computer: Rock -> Player Wins

```

```

----- FINAL SCOREBOARD -----
Round   Player Move   Computer Move   Result
1       Rock         Paper          Computer Wins
2       Paper        Rock           Player Wins
3       Scissors      Scissors       Draw
4       Rock         Paper          Computer Wins
5       Paper        Rock           Player Wins
-----
Wins: 2 | Losses: 2 | Draws: 1 | Win % = 40.0%

```

Problem 2 — Palindrome Checker (3 Approaches)

Problem Statement

The QA Text Verification Toolkit. The same text must be verified by three independent approaches — iterative comparison, recursion and array reversal — and all three must agree on the result.

Approach

- Iterative: two pointers start at both ends and move inward, returning false on the first mismatch. Runs in $O(n)$ time and $O(1)$ extra space.
- Recursive: compares the first and last characters, then recurses on the substring between them. The base case is a string of length 0 or 1.
- Array reversal: the string is copied into a char array, filled backwards into a second array, and the rebuilt string is compared with the original using equals().
- main() prints all three verdicts side by side and confirms explicitly that the approaches agree.

Java Program — PalindromeChecker.java

```
/**
 * Problem 2 - Palindrome Checker, 3 Approaches (QA Text Verification Toolkit)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class PalindromeChecker {

    /* Approach 1 - iterative two-pointer comparison. */
    public static boolean isPalindromeIterative(String text) {
        int left = 0, right = text.length() - 1;
        while (left < right) {
            if (text.charAt(left) != text.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }

    /* Approach 2 - recursion: compare the ends, then shrink the substring. */
    public static boolean isPalindromeRecursive(String text) {
        if (text.length() <= 1) return true;
        if (text.charAt(0) != text.charAt(text.length() - 1)) return false;
        return isPalindromeRecursive(text.substring(1, text.length() - 1));
    }

    /* Approach 3 - convert to a char array, reverse it, compare with the original. */
    public static boolean isPalindromeArrayReversal(String text) {
        char[] chars = text.toCharArray();
        char[] reversed = new char[chars.length];
        for (int i = 0; i < chars.length; i++) {
            reversed[i] = chars[chars.length - 1 - i];
        }
        return text.equals(new String(reversed));
    }

    private static String label(boolean isPalindrome) {
        return isPalindrome ? "Palindrome" : "Not Palindrome";
    }

    public static void main(String[] args) {
        String[] inputs = { "madam", "hello", "racecar", "step" };

        for (String text : inputs) {
            boolean a = isPalindromeIterative(text);
            boolean b = isPalindromeRecursive(text);
            boolean c = isPalindromeArrayReversal(text);

            System.out.println("Input: \"" + text + "\"");
            System.out.println(" Iterative: " + label(a)
                + " | Recursive: " + label(b)
                + " | Array Reversal: " + label(c));
            System.out.println(" All three approaches agree: " + ((a == b) && (b == c)));
            System.out.println();
        }
    }
}
```

```
    }  
  }  
}
```

Output

```
Input: "madam"  
  Iterative: Palindrome | Recursive: Palindrome | Array Reversal: Palindrome  
  All three approaches agree: true  
  
Input: "hello"  
  Iterative: Not Palindrome | Recursive: Not Palindrome | Array Reversal: Not Palindrome  
  All three approaches agree: true  
  
Input: "racecar"  
  Iterative: Palindrome | Recursive: Palindrome | Array Reversal: Palindrome  
  All three approaches agree: true  
  
Input: "step"  
  Iterative: Not Palindrome | Recursive: Not Palindrome | Array Reversal: Not Palindrome  
  All three approaches agree: true
```

Problem 3 — BMI Calculator for a Team

Problem Statement

The Corporate Wellness Program. Height (m) and weight (kg) are recorded for a team of 10 employees; the system must compute each BMI, classify the health status, and present everything in a clean table for the wellness report.

Approach

- Two parallel double arrays hold the heights and weights. Values are generated with a seeded Random over realistic ranges (1.50–1.95 m, 45–100 kg) so the demo runs instantly and reproducibly.
- BMI is computed as $\text{weight} / (\text{height} * \text{height})$ for each person.
- `getBmiStatus()` maps the value to the four bands using an if / else-if chain: below 18.5 Underweight, below 25 Normal, below 30 Overweight, otherwise Obese.
- `printWellnessReport()` uses `System.out.printf()` with fixed column widths so the table stays aligned, and the two sample cases from the problem statement are verified at the end.

Java Program — BmiCalculator.java

```
import java.util.Random;

/**
 * Problem 3 - BMI Calculator for a Team (Corporate Wellness Program)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class BmiCalculator {

    /* Classifies a BMI value into the standard WHO health bands. */
    public static String getBmiStatus(double bmi) {
        if (bmi < 18.5) return "Underweight";
        else if (bmi < 25.0) return "Normal";
        else if (bmi < 30.0) return "Overweight";
        else return "Obese";
    }

    /* Prints the full wellness report table for the whole team. */
    public static void printWellnessReport(double[] heights, double[] weights) {
        System.out.printf("%-8s %-12s %-12s %-10s %-12s\n",
            "Person", "Height (m)", "Weight (kg)", "BMI", "Status");
        System.out.println("-----");

        for (int i = 0; i < heights.length; i++) {
            double bmi = weights[i] / (heights[i] * heights[i]);
            System.out.printf("%-8d %-12.2f %-12.1f %-10.2f %-12s\n",
                (i + 1), heights[i], weights[i], bmi, getBmiStatus(bmi));
        }
        System.out.println("-----");
    }

    public static void main(String[] args) {
        int teamSize = 10;
        double[] heights = new double[teamSize];
        double[] weights = new double[teamSize];

        Random random = new Random(7); // fixed seed -> reproducible demo run
        for (int i = 0; i < teamSize; i++) {
            heights[i] = 1.50 + random.nextDouble() * 0.45; // 1.50 m .. 1.95 m
            weights[i] = 45.0 + random.nextDouble() * 55.0; // 45 kg .. 100 kg
        }

        System.out.println("WELLNESS CAMP - BMI REPORT");
        System.out.println();
        printWellnessReport(heights, weights);

        // Verification against the sample cases given in the problem statement.
        System.out.println();
        System.out.println("Sample verification:");
        double bmi1 = 70 / (1.75 * 1.75);
        double bmi2 = 90 / (1.60 * 1.60);
        System.out.printf("Person 1 - Height: 1.75 m, Weight: 70 kg -> BMI: %.2f | Status: %s\n",
            bmi1, getBmiStatus(bmi1));
    }
}
```

```
        System.out.printf("Person 2 - Height: 1.60 m, Weight: 90 kg -> BMI: %.2f | Status: %s\n",
                           bmi2, getBmiStatus(bmi2));
    }
}
```

Output

WELLNESS CAMP - BMI REPORT

Person	Height (m)	Weight (kg)	BMI	Status
1	1.83	86.2	25.77	Overweight
2	1.66	94.4	34.37	Obese
3	1.82	64.4	19.46	Normal
4	1.55	91.7	37.97	Obese
5	1.54	96.1	40.65	Obese
6	1.79	88.2	27.55	Overweight
7	1.77	71.0	22.79	Normal
8	1.61	66.0	25.34	Overweight
9	1.85	57.1	16.74	Underweight
10	1.64	92.7	34.42	Obese

Sample verification:

Person 1 - Height: 1.75 m, Weight: 70 kg -> BMI: 22.86 | Status: Normal
Person 2 - Height: 1.60 m, Weight: 90 kg -> BMI: 35.16 | Status: Obese

Problem 4 — First Non-Repeating Character

Problem Statement

The Unique Letter Hunt Mini-Game. A player types any word or sentence and the app must instantly find the first character that appears exactly once in the whole input.

Approach

- A 256-slot int array is used as an ASCII frequency table, so counting needs no HashMap and lookups are $O(1)$.
- First pass: increment the count for every character in the string.
- Second pass: scan left to right and return the first character whose count is exactly 1 — this early exit is what makes the result the **first** unique character rather than any unique character.
- '\0' is returned as a sentinel when every character repeats, and main() turns that into the "No Non-Repeating Character Found" message.

Java Program — FirstNonRepeatingCharacter.java

```
/**
 * Problem 4 - First Non-Repeating Character (Unique Letter Hunt Mini-Game)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class FirstNonRepeatingCharacter {

    /* Returns the first character with frequency 1, or '\0' when none exists. */
    public static char findFirstNonRepeatingChar(String text) {
        int[] frequency = new int[256]; // ASCII frequency table

        for (int i = 0; i < text.length(); i++) {
            frequency[text.charAt(i)]++;
        }
        for (int i = 0; i < text.length(); i++) {
            if (frequency[text.charAt(i)] == 1) {
                return text.charAt(i); // early exit on the first hit
            }
        }
        return '\0'; // sentinel: nothing unique
    }

    public static void main(String[] args) {
        String[] inputs = { "swiss", "aabbcc", "programming", "step sem 3" };

        for (String text : inputs) {
            char result = findFirstNonRepeatingChar(text);
            System.out.println("Input: \"" + text + "\"");
            if (result == '\0') {
                System.out.println(" No Non-Repeating Character Found");
            } else {
                System.out.println(" First Non-Repeating Character: '" + result + "'");
            }
        }
    }
}
```

Output

```
Input: "swiss"
First Non-Repeating Character: 'w'
Input: "aabbcc"
No Non-Repeating Character Found
Input: "programming"
First Non-Repeating Character: 'p'
Input: "step sem 3"
First Non-Repeating Character: 't'
```

Problem 5 — Reverse Customer Name

Problem Statement

The Customer Identity Verification System. A banking module generates a reversed copy of a customer's name for an internal testing exercise, and must print both the original and the reversed name while leaving the original data unchanged.

Approach

- The name is copied into a char array with `toCharArray()`, so the original String object is never touched — Strings are immutable in Java, which guarantees the requirement.
- A `StringBuilder` is filled by walking the array from the last index down to 0.
- The method returns the rebuilt string, and `main()` prints the original and reversed names for several customers.

Java Program — ReverseCustomerName.java

```
/**
 * Problem 5 - Reverse Customer Name (Customer Identity Verification System)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class ReverseCustomerName {

    /* Builds and returns the reverse of the name; the original is never modified. */
    public static String reverseCustomerName(String customerName) {
        char[] characters = customerName.toCharArray();
        StringBuilder reversed = new StringBuilder();

        for (int i = characters.length - 1; i >= 0; i--) {
            reversed.append(characters[i]);
        }
        return reversed.toString();
    }

    public static void main(String[] args) {
        String[] customers = { "Sunil", "Gursimran", "Anita Rao" };

        for (String name : customers) {
            String reversed = reverseCustomerName(name);
            System.out.println("Original Name: " + name);
            System.out.println("Reversed Name: " + reversed);
            System.out.println();
        }
    }
}
```

Output

```
Original Name: Sunil
Reversed Name: linuS

Original Name: Gursimran
Reversed Name: narmisruG

Original Name: Anita Rao
Reversed Name: oaR atinA
```